Sealed Execution: Letting LLM Coding Agents Use Credentials Without Seeing Them

Mihir Gupta Stephen Keehn
Generalized Labs
https://github.com/Generalized-Labs/ironrun

Abstract

LLM coding agents run shell commands, and whatever those commands print becomes model context. A credential that surfaces in output—via `printenv`, a verbose `curl`, or a stack trace—reaches the model provider and local transcripts and must be rotated. We present ironrun, an open-source runtime that injects secret values into child processes as environment variables or temporary files and filters known values and supported encodings from their output before it reaches the agent. ironrun combines (i) value-blind agent interfaces exposed over the Model Context Protocol, (ii) human authorization tiers bound to a single agent session, environment, and expiry, (iii) a streaming known-value redactor with whitespace-tolerant, fragment, and alignment-aware encoding matching under a bounded hold-back, and (iv) fail-closed containment—network and syscall isolation, process-group lifetime control, CI fork gating—with a hash-chained audit log. We report an adversarial self-audit of the 16.6 kLOC Go implementation that confirmed 49 distinct defects—2 critical and 6 high severity, 44 now fixed—among them a YAML-injection path that let an agent smuggle commands past human review and a CLI command that printed the vault root key. On a corpus of 21 output transformations × 6 credential formats run through the real execution path, the number of cases that leak at least half of a secret falls from 52 to 28; for transformations produced by ordinary tools it falls from 21 of 72 to 3 of 72, all three involving a 12-byte password below the fragment threshold. The residue includes these three accidental cases and deliberate transformations not covered by the registered patterns. Overhead is about 0.1 ms per command, and the redactor sustains 14–36 MB/s. We argue that redaction is a safety net for *accidental* disclosure, while authorization and isolation must bound *deliberate* exfiltration.

1 Introduction

Coding agents such as Claude Code, Codex, and Cursor operate by issuing tool calls—most importantly, shell commands—and reading their output. That output is appended to the model's context window, transmitted to the model provider on every subsequent turn, and typically persisted by the agent host as a local transcript (Claude Code, for example, stores JSONL session logs under `~/.claude/projects/`). Anything a command prints is therefore *published*: to the provider, to the transcript store, and to anyone who later reads, shares, or screen-records the session.

Credentials reach output through utterly ordinary debugging. An agent asked to fix a failing integration test may run `printenv` to check configuration, `cat .env` to inspect it, `curl -v` to reproduce an HTTP error (which echoes the `Authorization` header), or simply rerun a test whose failure message embeds the database URL. Each of these discloses a secret the human never intended to share, and because transcripts cannot be retracted from a provider's logs, the only safe response is rotation. Practitioners report this happening repeatedly in normal use [14, 56], and a study of 17,022 agent skills reports debug-output exposure in 1,007 of 1,371 vulnerability issues (73.5%), where agent frameworks feed output into model context [16]. This percentage describes that study's issue category, not a population-wide rate of agent credential leakage. Once a value has entered context it also resurfaces in places transcript scrubbing cannot reach, such as reasoning traces [59].

Existing tools each cover part of the problem. Secret managers' `run` wrappers inject values into a child's environment [2, 24, 41]; 1Password's also conceals values in the child's output, but none of them mediates *which* commands an agent may run with which secrets. Agent-host deny rules that hide `.env` files do not stop commands that read the file indirectly [6, 25, 53]. Credential brokers control *who may authenticate* [1, 12], not *what a command prints after authenticating*. Proxy-substitution designs never give the process the real value at all, but apply only to HTTP(S) traffic through a TLS-terminating proxy [7, 26, 40]. CI runners have long masked known secrets and their encodings in job logs [31, 32], and agent-side tools have recently begun to scrub tool output by literal matching [27, 61]. What is missing is a design—and an evaluation—that combines these at the agent's tool boundary for arbitrary command-line programs: database clients, test runners, `kubectl`, `terraform`.

Insight. The agent needs the *effect* of a credential—a passing test, a successful deploy—not its *value*. If the same component that resolves and injects secrets also mediates the command's output, then redaction becomes a problem over *known* values rather than a detection problem over unknown ones. Known-value redaction is deterministic over a fixed pattern set and can be extended systematically to the encod-

ings that ordinary tools apply. Coverage still depends on registration thresholds and overlapping-match behavior; it is not a universal confidentiality guarantee.

Contributions.

- A threat model for credential disclosure through agent tool output that separates *accidental* disclosure, *prompt-injected* agents acting through their tools, and hostile same-user code (Section 3).
- The design and implementation of ironrun, an MIT-licensed local runtime comprising value-blind MCP interfaces, session-bound tiered authorization with in-place continuation, a streaming redactor with a proved chunking-invariance property, and layered containment (Sections 4 and 5).
- An adversarial self-audit of the implementation, its findings, and the lessons they teach about where such systems fail: in the text-rendering and state surfaces around the cryptography, not in the cryptography (Section 6).
- To our knowledge the first adversarial evaluation of known-value redaction at an agent's tool boundary: a 126-case leak corpus driven through the real execution path and attacked with a decoder battery, plus false-positive and overhead measurements, with an open harness (Section 7).

2 Background

Agent tool loops. An agent host repeatedly sends the conversation to a model, receives tool calls, executes them locally, and appends results. The Model Context Protocol (MCP) [52] standardizes how a host discovers and invokes local tool servers over stdio. Every byte a tool returns becomes model input, is retained for the rest of the session, and is commonly written to disk by the host.

Why output is the hard channel. Secrets can leave through three channels: *inputs* the agent writes (a value typed into a command line), *outputs* a command prints, and *side effects* (network, files). The first is solvable by never giving the agent the value; the third is the domain of sandboxes. The second is the subtle one: an agent that has never seen a credential can still cause it to be printed, merely by running the program that uses it. Rotation is the only remedy after the fact.

Scenarios. Three representative accidents motivate the design: (1) a connection failure prints a DSN with an embedded password; (2) `curl -v -u deploy:$TOKEN` prints `Authorization: Basic` followed by the base64 of `deploy:<token>`; (3) `docker compose config` prints the interpolated environment. In (2) the token never appears literally—it appears base64-encoded *at a byte offset that depends on the username*, a case that matters in Section 7.

3 Threat Model

Principals. The *operator* is the human developer; they are trusted, author the policy, and approve access. The *agent* is the model together with its host; it is trusted to act on the operator's behalf but may be mistaken or manipulated. *Child commands* are the operator's own build, test, and deploy programs, run on the agent's request. The *model provider* receives everything in context and is outside the operator's control.

Adversaries. We distinguish three:

A1 Accidental disclosure. An honest agent runs ordinary commands that happen to print secrets. This is the common case and the primary target.

A2 Manipulated agent. A prompt-injected or misaligned agent [33] tries to move a secret into its context or off the machine, but acts only through its tools: MCP calls, its native shell, and file edits.

A3 Hostile same-user code. Arbitrary native code running as the operator's UID. It can read the user's keychain, attach to processes where the OS permits, and rewrite user-owned files. A3 is out of scope; containing it requires a separate OS user, container, or VM.

Goals.

G1 Value-blindness. No ironrun interface reachable by the agent—MCP tools, CLI, local API, daemon, audit log—returns a secret value or key material.

G2 Output confidentiality. When an authorized child prints a secret it was given, in literal form or a common encoding, the agent receives a placeholder instead.

G3 Least authority. The agent obtains secret-bearing execution only for commands and environments a human approved, bound to one agent session and an expiry; revocation takes effect on the next call.

G4 Containment. By default a secret-bearing command has no network, cannot trace other processes, cannot dump core, and cannot outlive its time budget.

G5 Accountability. Every run leaves a tamper-evident record of what ran with which secret *names*.

Non-goals. ironrun does not attempt to stop an authorized process that *deliberately* transforms a secret with an arbitrary function before printing it (Observation 1); does not defend against A3; does not remove data from provider logs; and does not guarantee physical erasure of temporary file secrets on SSDs or copy-on-write filesystems.

A note on A2 and the native shell. Agent hosts typically give the agent its own unrestricted shell in addition to any MCP server. ironrun removes secrets from the agent's ambient environment and from the repository, which defeats A1 and casual A2 (`printenv`, `cat .env`). But a determined A2 with an unrestricted same-user shell approaches A3: on macOS, for instance, a keychain item created through `/usr/bin/security` is readable by any process that invokes

Tool	Returns (never a value)
`list_commands`	IDs and argv of approved commands
`run_sealed`	exit code, duration, redacted stdout/stderr
`request_workspace_access`	request ID for a trusted session
`workspace_status`	grant state, environment names
`request_lease` / `lease_status`	lease request / state
`revoke_own_lease`	revocation of the caller's own lease
`propose_command`	staged proposal ID (runs nothing)
`request_secret`	request ID (no value field)
`claim_capsule`	success flag; stores ciphertext's value locally
`list_environments`	names and key names only
`validate_policy`	provider and command count

Table 1: The agent-facing MCP surface. A guard test fails the build if any tool result carries vault key material, and removed tools (e.g. an earlier `share_environment`) are registered as banned.

the same binary. Deployments that face A2 must therefore pair ironrun with the host's own command restrictions or sandbox, or run ironrun's state under a separate user. We return to this in Section 8.

4 Design

Figure 1 shows the architecture. All entry points—CLI, MCP server, TUI, and a local Unix-socket API—converge on a single execution core, so that authorization, resolution, containment, redaction, and audit cannot diverge between surfaces.

4.1 Value-blind interfaces

The agent-facing MCP server exposes twelve tools (Table 1). Their schemas are designed so that no field can carry a secret value in either direction. `request_secret`, for example, accepts only a declared alias and a reason; the human fulfils it through masked local input. If a workflow requires pasting through chat, the operator instead produces an *encrypted capsule* (`ir1....`): AES-256-GCM ciphertext under a per-project capsule key, bound by its authenticated data to the project and by its payload to the requesting MCP session, request, alias, and a ≤10-minute expiry, and usable once. The plaintext never enters the transcript.

4.2 Authorization tiers

ironrun offers four ways for an agent to obtain secret-bearing execution, ordered from strictest to most convenient:

1. **Strict commands.** The policy file lists exact argv vectors and the secrets each receives. The agent calls `run_sealed` with a command ID; no shell is interposed, so injected values cannot be re-expanded, and argv[0] may not be a shell. The policy is re-read on every call.
2. **Leases.** With `require_agent_leases`, a strict command also needs an unexpired, unrevoked lease bound to the MCP session, environment, and command set (default 1 h, maximum 24 h).
3. **Trusted workspace sessions.** For everyday development, the operator grants one MCP session arbitrary-argv execution against one project environment for a bounded time (default 2 h). `staging` and `prod` require separate grants.
4. **Proposals.** An agent may stage a new strict command with `propose_command`; it is inert until the operator approves it, at which point it is merged into the policy.

Session identifiers are 64-bit values from `crypto/rand`, minted per MCP server process and never accepted from the agent, so a grant cannot outlive the server instance it was issued to.

Continuation. A blocked call does not fail; it creates (or reuses) a deduplicated, value-blind request and waits on persisted state, re-validating authorization every 100 ms until the request's deadline (Fig. 2). When the operator approves in the TUI, the original call resumes. This removes the round-trip in which an agent asks the human to "approve and tell me when done"—a round-trip that, in practice, encourages humans to paste secrets into chat.

4.3 Resolution and storage

Secrets come either from an existing manager through its CLI (1Password, HashiCorp Vault, Doppler, Infisical), from an owner-only env file outside the repository, or from ironrun's own encrypted vault (Fig. 3). The vault keeps a random 32-byte root key in the OS credential manager. Each environment (a *scope*) is sealed under a fresh 32-byte data key on every write; the data key is wrapped by the root key with AES-256-GCM, and both encryptions bind version, purpose, project, and scope in their authenticated data, so ciphertexts cannot be moved between environments or projects. The whole manifest carries an HMAC-SHA256 under the root key and is committed by write-temp, fsync, rename, and directory fsync. Only names, kinds, and timestamps are stored unencrypted, under `.ironrun/` in the project.

File secrets (e.g. a service-account JSON) are materialized only for the duration of a run, in a fresh 0700 directory outside the repository, with `O_EXCL` 0600 files and validated basenames; only the path is injected, and the contents are registered for redaction.

4.4 Streaming known-value redaction

Each of the child's output streams passes through its own redacting writer. Let V be the set of injected values (and file-secret contents) of at least 4 bytes. The writer registers a pattern set $P = V \cup \bigcup_{v \in V, |v| \geq 8} \mathrm{Enc}(v)$, where Enc derives the encodings that ordinary tools apply: base64 (standard and URL alphabets, padded and raw), lower- and upper-case hex, percent-encoding in the styles of Go, JavaScript, and

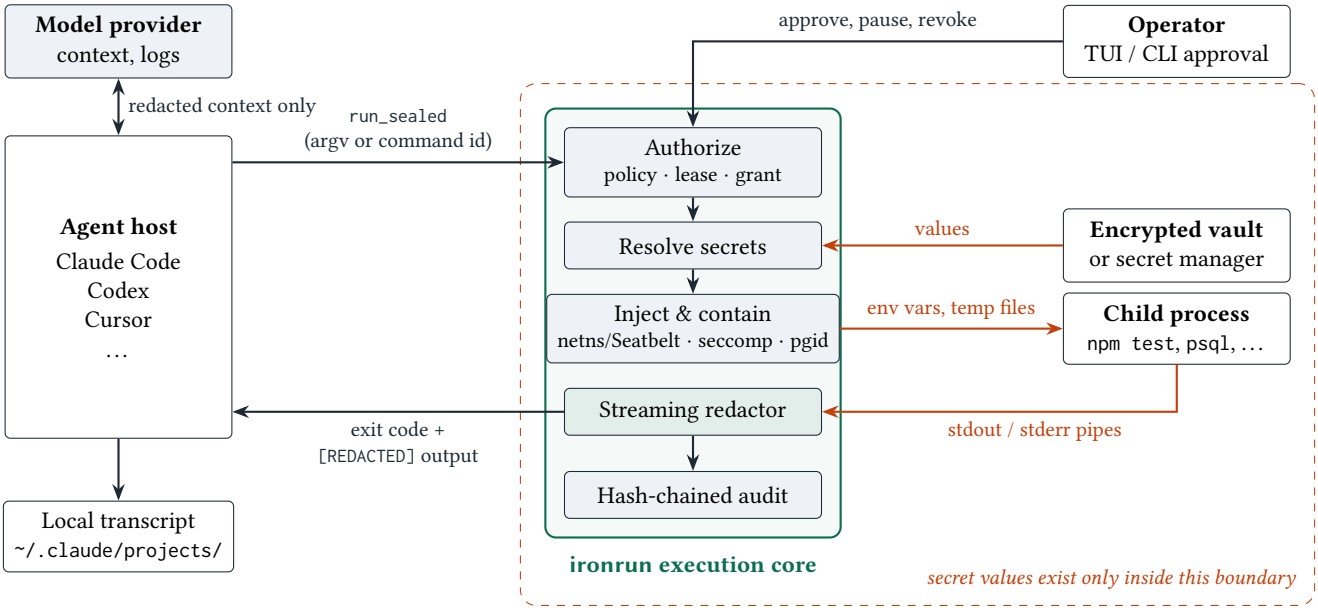


Figure 1: ironrun architecture. Values (orange) move from the vault into the child and from the child's pipes into the redactor; they never cross to the agent. The agent invokes one tool, receives an exit code and redacted output, and cannot approve its own requests.

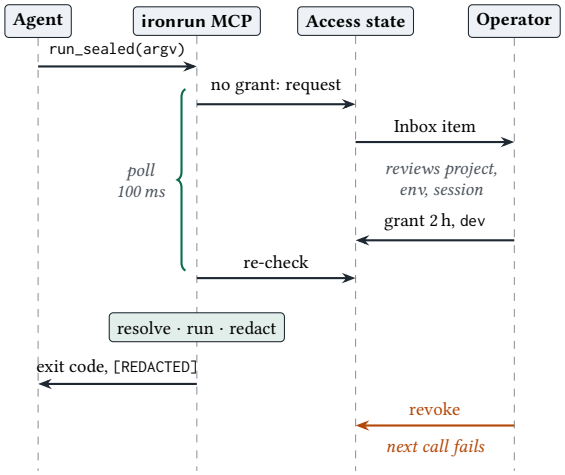


Figure 2: Request, approval, and in-place continuation. The waiting call is never restarted, and approval happens only on the operator's side.

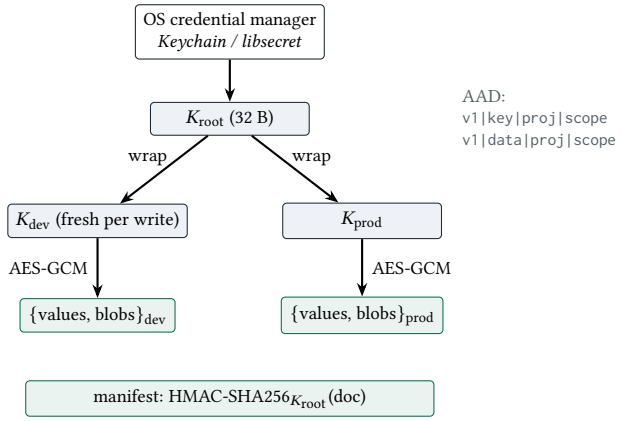


Figure 3: Vault key hierarchy. Rewriting a scope rotates its data key; AAD binds every ciphertext to its project and scope.

Python (with upper- and lower-case escapes), JSON string escaping, second-level nestings (e.g. base64 of base64), base64 wrapped at 64 and 76 columns, and—crucially for the Basic-auth scenario—the *alignment cores* of base64 at byte offsets 1 and 2 (Section 4.4.1).

Matching tiers. At each buffer position the writer tries, in order:

1. **Exact**, longest pattern first, using a first-byte index.
2. **Gap-tolerant**: the pattern's bytes appear in order with interleaved whitespace or control bytes (≤ `0x20`, `0x7F`), which covers terminal line wrapping, `\r \n` injection, UTF-16LE null interleaving, and one-character-per-line output. The consumed span is capped at the global span m defined below.
3. **Fragment**: for patterns with 24 to 65,536 non-whitespace bytes, every 12-byte window of the pattern's whitespace-free projection is indexed. Twelve output bytes that match a window, in order and with any gap bytes between them, are extended forward over the rest of the pattern and redacted. This catches truncated and partial prints (one line of a PEM key) and line-oriented dumps (`xxd`, `od -c`) whose offset columns cut a value into per-line pieces.

A match is replaced by `[REDACTED]`; unmatched bytes are emitted only once no pattern could still begin at them.

Bounded hold-back. Let $m = \max(\{1\} \cup \{2|p| + 32 : p \in P\})$ bound the span any tier may consume, including when P is empty. The writer withholds the last $m - 1$ bytes of its buffer until more input arrives or the stream is flushed (Algorithm 1 and Fig. 4).

Property 1 (Chunking invariance). *For fixed patterns and*

Algorithm 1 Streaming redaction (one output stream)

```
1:  procedure Write(x)
2:      B ← B ∥ x; Scan(m − 1)
3:  procedure Flush
4:      Scan(0)
5:  procedure Scan(h)
6:      while |B| > h do
7:          i ← 0
8:          while i < |B| − h and Match(B, i) = 0 do
9:              i ← i + 1
10:         ℓ ← 0
11:         if i < |B| − h then
12:             ℓ ← Match(B, i)
13:         Emit(B[0:i])
14:         if ℓ = 0 then
15:             B ← B[i:]; break
16:         Emit([REDACTED]); B ← B[i + ℓ:]
17: function Match(B, i)                    ▷ bytes consumed, ≤ m
18:     return first nonzero of Exact, GapTolerant, Fragment
    at B[i:]
```

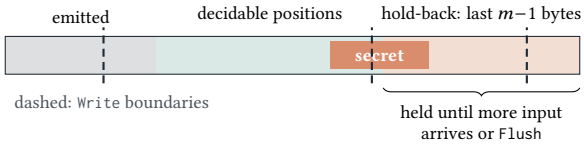


Figure 4: The scanner only decides positions that have at least $m-1$ bytes of lookahead, so a secret that straddles one or more `Write` boundaries is still seen whole.

matching indexes, complete successful underlying writes, and a single final `Flush`*, any partition of input* x *into* `Write` *calls emits the same output as* `Write`(x) *followed by* `Flush`*.*

Proof sketch. Scan decides position i only if $|B| - i > m - 1$, i.e. at least m bytes starting at i are buffered. Every tier's decision depends on at most m bytes from i: exact matching reads $|p| < m$ bytes; gap-tolerant matching accepts spans of at most m; fragment lookup and extension are explicitly limited to m bytes. Hence Match(B, i) depends only on bytes already present and is identical to its value on the complete input. Positions are decided left to right in the same order in both executions, and `Flush` decides the remaining positions with the full suffix present. Induction on decided positions gives equal output. □

The cost of this guarantee is latency: up to $m - 1$ bytes (a few hundred bytes for typical tokens) are delayed until the next write or process exit. This bounds buffered bytes, not wall-clock delay: a quiet interactive process may leave output buffered until it prints more data or exits. Output order is preserved.

4.4.1 Base64 alignment

Base64 maps each 3-byte group to 4 characters. When a secret s of n bytes is encoded as part of a larger message, the characters that depend *only* on s are determined by s's offset modulo 3. With $k \in \{0, 1, 2\}$ bytes of preceding context, they are

$$\mathrm{core}_k(s) = \mathrm{b64}(0^k \| s)\big[\lceil 8k/6 \rceil \,:\, \lfloor 8(k+n)/6 \rfloor\big],$$

where the trailing bound also drops the final character, which mixes bits of s with whatever follows. Registering only $\mathrm{b64}(s)$, as ironrun v0.5.0 did, matches only when $k = 0$ and $n \equiv 0 \pmod 3$ and the encoder happens to stop at s; HTTP Basic authentication (`user:token`) and `printenv | base64` both violate this two times out of three. Registering the three cores closes the gap for any surrounding context. The technique is not new—the GitHub Actions runner has registered shifted base64 forms since 2019 [31]—and its absence from v0.5.0 is one of the audit findings of Section 6.

4.5 Containment

Redaction protects the output channel; the following layers bound side effects, and all of them fail closed:

- **Network.** Secret-bearing commands get no network unless the policy opts out. On Linux ironrun enters fresh user and network namespaces [46, 47] (`CLONE_NEWUSER|CLONE_NEWNET` with an identity UID/GID map, so no privilege is needed); on macOS it wraps the child in a Seatbelt [13] profile that denies only `network*`, invoking `/usr/bin/sandbox-exec` [8] by absolute path. Unsupported platforms refuse to run.
- **Syscalls.** On Linux the child is started through a re-exec shim that sets `NO_NEW_PRIVS`, installs a seccomp-BPF filter [68] that checks the architecture, rejects the x32 ABI, and returns `EPERM` for `ptrace`, `process_vm_readv/writev`, `kcmp`, `perf_event_open`, `bpf`, and `userfaultfd`, then sets `RLIMIT_CORE=0` and execs the target in place.
- **Lifetime.** The child runs in its own process group; on timeout, cancellation, or a signal to ironrun, the whole group is killed, and any group member left after the child exits is killed too. Descendants that detach with `setsid` can escape this group and retain injected values; this is not whole-process-tree containment. Temporary file secrets are removed during normal error, cancellation, timeout, and success cleanup, with stale-directory recovery after crashes.
- **Environment.** The inherited environment is stripped of loader, shell-startup, interpreter, and tool hijack variables (`LD_*`, `DYLD_*`, `BASH_ENV`, `NODE_OPTIONS`, `PYTHON*`, `GIT_CONFIG_*`, `GOFLAGS`, …) before secrets are added.
- **CI.** In GitHub Actions, GitLab CI, CircleCI, and Jenkins, runs triggered by fork pull requests (and `pull_request_target`, the "pwn request" pattern [44, 49], without an explicit operator flag) are refused before any secret is resolved.
- **Operator-only switches.** Every control that weakens protection (seccomp, core-dump seal, entropy scan, `pull_request_target`) is a command-line flag, never an environment variable, because the environment is reachable by the agent.

4.6 Audit

Each run appends a JSONL record—command, argv with all registered patterns scrubbed, secret names and kinds, redaction count, exit status, sandbox outcome—whose hash covers the previous record's hash, in the style of Schneier and Kelsey [66]. `ironrun audit verify` replays the chain, and a broken chain makes subsequent runs refuse rather than continue unaudited. The chain is tamper-evident for edits and reordering; without an external anchor or a Merkle structure [19] it cannot detect truncation of the tail (Section 8).

5 Implementation

ironrun is a single static Go binary (17.3 kLOC of production code, 11.5 kLOC of tests across 431 test functions, fuzzing of the redactor) built with `CGO_ENABLED=0`. The MCP server uses `mcp-go`. Platforms are macOS and Linux on amd64 and arm64; Windows is unsupported because its secret-entry paths would require plaintext console input. Builds use `-trimpath`, commit-date timestamps, and pinned modules. Releases carry SHA-256 checksums, an SBOM, and keyless Sigstore bundles [57]. The installer and GitHub Action verify checksums and bundles; npm verifies archive and binary hashes against its bundled manifest, without independent Sigstore verification. CI runs OpenSSF Scorecard, `govulncheck`, and scheduled fuzzing.

6 Adversarial Self-Audit

Before release we audited v0.5.0 (16.6 kLOC) adversarially. Five reviewers each took one subsystem—redaction, execution and sandboxing, authorization and agent surfaces, storage and cryptography, and policy/CLI/distribution—and were instructed to report only defects backed by an executed probe or a line-level trace against the documented guarantees. After removing findings reported by more than one reviewer, 49 distinct defects remained (2 critical, 6 high, 16 medium, 25 low). We fixed 44, each with a regression test that fails on v0.5.0; the other five follow from the same-user threat boundary or are deliberate trade-offs, and are documented as limitations (Section 8). The reviewers were LLM-based code review agents directed and checked by the authors; we report this because the method is reproducible and because the class of findings is instructive. Table 2 summarizes the results.

Lesson 1: human-in-the-loop is only as strong as what the human sees. A1–A3 share a root cause: approval rendered agent-controlled text into a second language (YAML) by string concatenation, and showed it to the human through a third (a terminal). The reviewer saw a "reason"; the parser saw a command. The fix quotes every agent-derived scalar, flattens comments to a single line under YAML's definition of a line break (which includes NEL, U+2028, and U+2029), rejects multi-line text at the MCP boundary, makes pending proposals immutable, and shows argv with explicit element boundaries. The general rule is that an approval UI must display the *parsed* artifact that will be enforced, not the text that produced it.

Lesson 2: value-blindness is a property of every surface the agent can reach. The project had removed an MCP tool that returned the root key and added a guard test against its return—but the equivalent CLI command remained, and agents have shells. S1 and R1 are both instances of a value leaving through a surface designed for humans or CI. We now treat the CLI as agent-reachable and test it accordingly.

Lesson 3: encodings have structure. R2 is not a missing encoding but a missing *alignment*; R4 and R5 are formatting conventions of specific tools. Enumerating "base64" as a single transformation was the error.

Lesson 4: fail-closed tests can pass for the wrong reason. The macOS network test asserted a non-zero exit, which the sandbox produced by crashing the probe before it dialed (E3). The corrected test distinguishes "connection refused by policy" from "process failed".

7 Evaluation

We ask: **RQ1** how much of a secret reaches the agent under realistic and adversarial output transformations; **RQ2** how often benign output is redacted; **RQ3** what the runtime costs; and **RQ4** whether streaming behaves exactly like batch processing. All numbers come from the open harness in `paper/eval` on an Apple M3 Pro (macOS 26.4, Go 1.26.9).

Method (RQ1). We generate six synthetic credentials shaped like real formats: a Stripe-style key (32 B), a GitHub PAT (40 B), an AWS-style secret (40 B), a Postgres URL with an embedded password (75 B), a 12-byte password, and a JWT (113 B). For each of 21 transformations we write the transformed output to a file and have a real child (`cat`) print it through `runner.Run` with the secret injected—the same injection, pipe, registration, and redaction path a sealed run uses. We then attack the agent-visible output with a decoder battery: identity, gap stripping, case folding, reversal, ROT13, base64 and hex decoding at every alignment (with nesting and gunzip), URL and JSON unescaping, and hexdump column reassembly. *Recovered* is the longest contiguous run matched after case folding by any decoder; a case *leaks* if at least 50% of the secret matches. This measures case-insensitive contiguous disclosure, not necessarily a usable credential. A shorter recovered component can still expose the entire password inside a structured URL. Rates describe this deterministic corpus, not a real-world probability of leakage.

ID	Sev.	Finding	Resolution
Agent surfaces and authorization			
A1	High	Proposal approval spliced agent text (reason, env names/refs, v2 secret names) into `ironrun.yml` unescaped; a newline—or U+2028, which YAML also treats as a line break—smuggled extra commands past human review	fixed
A2	Med	Approval bound to a proposal ID whose content the agent could replace between display and approval	fixed
A3	Med	Approval display lost argv boundaries, passed control characters to the terminal, and hid v1 secret references in the TUI	fixed
A4	Med	A trusted session can run ironrun itself to grant or extend its own access	documented
A5	Low	`propose_command` and the local API used the startup policy, delaying revocation	fixed
Storage and cryptography			
S1	Crit	`ironrun env share` printed the vault root key to stdout, with no gate	fixed (removed)
S2	High	`env sync pull` overwrote the vault with unverified remote bytes (data loss, rollback)	fixed
S3	Med	Root key passed on the `security` command line (visible to process listings and EDR)	fixed
S4	Med	Audit log failed open on tamper detection; tail truncation undetected	fixed / documented
S5	Med	Equivalent remote URLs (HTTPS vs. SSH) changed project identity and locked users out	fixed
S6	Low	No anti-rollback for vault copies; lost metadata updates under concurrency	documented / fixed
Redaction			
R1	Crit	GitHub mask emission printed lines 2..*N* of multi-line secrets (PEM keys, service-account JSON) to job logs in cleartext	fixed
R2	High	Base64 matched only at offset 0 (Basic auth, docker `auth`, `printenv|base64` leaked)	fixed
R3	High	No fragment coverage for secrets over 1 KiB (partial prints of PEM/JSON file secrets leaked)	fixed
R4	Med	Hex dumps (`xxd`, `hexdump -C`, `od -c`) defeated hex coverage	fixed
R5	Med	Only Go's URL escapers covered; JSON-escaped forms missed (incl. in transcript scrubbing)	fixed
Execution and sandboxing			
E1	High	TTL not enforced when a grandchild held the output pipe; descendants outlived the run with secrets	fixed
E2	Med	A signal to ironrun left plaintext file secrets on disk and orphaned the child	fixed
E3	High	macOS network sandbox also denied `fork`, breaking most real commands; the macOS network test passed vacuously	fixed
E4	Med	`sandbox-exec` resolved through `PATH`, letting the environment disable isolation	fixed
E5	Low	Relative `workdir` resolved against the caller's cwd instead of the project root	fixed
E6	Low	Shell denylist bypassable by path variants (`/bin//sh`); env denylist gaps (`GOFLAGS`, `GIT_CONFIG_*`)	fixed
Policy, CLI, and distribution			
P1	Med	GitHub fork-PR detection compared a variable GitHub never sets, so fork runs were trusted; secrets were resolved and masks printed before the gate	fixed
P2	Med	Git hooks were installed where git does not read them (linked worktrees, `core.hooksPath`); hook scanning missed multi-line and binary values and non-active environments	fixed
P3	Med	The next stable release would fail before signing (missing tap token); the npm package could not be installed	fixed
P4	Low	Third-party actions in the signing workflow were pinned by mutable tag	fixed (SHA)
P5	Low	Dotenv and envfile parsing corrupted values and paths; policy `env:` keys unvalidated	fixed

Lower-severity items (exit-code reporting, truncation flags, backup files left by history purge, import-guard path handling) were also fixed and are listed in the changelog.

Table 2: Defects confirmed by the self-audit of v0.5.0.

Results (RQ1). Table 3 shows that v0.5.0 already handled literal prints, xtrace, URL parameters, aligned base64, wrapping, CRLF, UTF-16, and nested encodings, but leaked every secret under three *accidental* transformations—Basic auth, `printenv | base64`, and `xxd`—for the alignment and formatting reasons of Section 6. After the fixes, every accidental transformation is closed for all credentials of 24 bytes or more. The three remaining accidental cases all involve the 12-byte password, which is below the fragment threshold: a 60% prefix, a color escape in the middle of the value, and the `xxd` ASCII column, which prints it in two pieces.

Four deliberate transformations remain out of reach, and necessarily so:

Observation 1. *Known-value matching alone does not guarantee confidentiality against arbitrary transformations. An authorized process can disclose a secret through an invertible representation if the emitted representation avoids every registered match. Being unregistered is not sufficient: a transformation can still produce a registered pattern (for example, reversal of a palindrome).*

Redaction is therefore a defense against A1, not against an A2 that can choose the program. Against A2, ironrun relies on the strict tier with host-enforced protection of executable code and inputs, on network isolation (the transformed secret cannot leave except through output), and on the warn-only entropy scan, which flagged 21 of the 28 residual leaks, at the cost of 6 false alarms among the 98 correctly redacted cases. Compression is the instructive exception: gzip output is binary and therefore unreadable in a transcript, yet trivially decodable, which is why it is in the corpus.

RQ2: false positives. We run the redactor over 1.1 MB of benign text (the repository's own sources and documentation, plus 1,000 log lines that share structure with a connection URL) with each credential and its full encoding set registered (Table 4). Random-format credentials produced no false positives. The structured Postgres URL produced 1,002 both before and after the fixes: its non-secret 12-byte windows (`db.internal:5432/app`) occur in ordinary logs and trigger the fragment tier. Over-redaction is safe but degrades output; we discuss mitigations in Section 8.

RQ3: overhead. Spawning `true` directly takes a median 1.62 ms; through `runner.Run` with one secret it takes 1.68 ms (+0.06 ms), and with macOS network isolation 7.43 ms, domi-

Transformation	Class	v0.5.0	Fixed
`printenv` line	acc.	0/6	0/6
`set -x` trace	acc.	0/6	0/6
JSON (Go encoder)	acc.	1/6	**0/6**
URL query parameter	acc.	0/6	0/6
HTTP Basic auth	acc.	6/6	**0/6**
k8s Secret (base64)	acc.	0/6	0/6
`printenv | base64`	acc.	6/6	**0/6**
`xxd` hexdump	acc.	6/6	**1/6**
ANSI-highlighted	acc.	1/6	1/6
pty wrap (CRLF)	acc.	0/6	0/6
UTF-16LE	acc.	0/6	0/6
truncated (first 60%)	acc.	1/6	1/6
hex	del.	0/6	0/6
base64 at offset 1	del.	6/6	**0/6**
double base64	del.	0/6	0/6
gzip | base64	del.	6/6	6/6
one char per line	del.	0/6	0/6
split with separator	del.	1/6	1/6
reversed	del.	6/6	6/6
ROT13	del.	6/6	6/6
upper-cased	del.	6/6	6/6
Accidental		**21/72**	**3/72**
All		**52/126**	**28/126**

Table 3: Secrets (of 6) of which ≥50% is recoverable from agent-visible output, before and after the audit fixes. *acc.*: produced by ordinary tools; *del.*: deliberate.

Credential	Patterns†	FP (v0.5.0)	FP (fixed)
Stripe key	18	0	0
GitHub PAT	18	0	0
AWS secret	18	0	0
Postgres URL	22	1,002	1,002
password (12 B)	14	0	0
JWT	22	0	0

Table 4: Redactions on 1.1 MB of benign text (all are false positives). †Registered patterns per credential after the fixes.

nated by `sandbox-exec` start-up (n=200, fixed version). Registering the new encodings raises the patterns per 40-byte credential from 15 to 18 and lowers single-secret throughput from 56 to 36 MB/s; both remain far above interactive output rates. Provider look-ups add their own latency (e.g. a 1Password CLI call). Figure 5 reports redactor throughput on 4 MiB of log text written in 32 KiB chunks.

RQ4: streaming equivalence. Property 1 is exercised by a differential test that feeds 3,000 random inputs (secrets, their encodings, gap-interleaved and fragmentary forms) through the writer in chunks of 1–13 bytes and compares against a single write; no divergence was observed on v0.5.0 or after the fixes. The repository additionally fuzzes the writer continuously in CI.

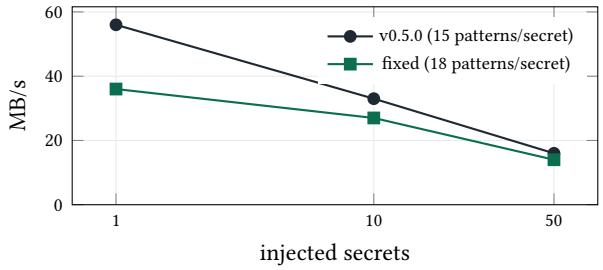


Figure 5: Redactor throughput. Terminal-scale output is far below either curve.

8 Discussion and Limitations

Substitution is stronger than redaction. In proxy-substitution designs the child holds only a placeholder and a proxy inserts the real value into outbound HTTP(S) requests [7, 20, 40, 58], so no output transformation can reveal it. Where that applies, it should be preferred. It does not apply to the many tools that use credentials outside HTTP—database wire protocols, SSH, signing keys, file-based service-account credentials—which is the space ironrun targets; the two compose.

Redaction is a safety net, not a sandbox. Observation 1 is fundamental. ironrun's strongest configuration—strict commands, leases, and no network—restricts argv. Restricting what the program computes additionally requires protecting its code, dependencies, and inputs from agent edits. Trusted workspace sessions trade this away for convenience: the agent chooses argv, receives every entry of the environment in the child, and could deliberately encode a secret. We document trusted sessions as protecting context and logs from accidents, not as containment.

State integrity against A2. Authorization state (`.ironrun/`, `ironrun.yml`) lives in the project, where an agent with file-edit tools can write it, and a trusted session can run ironrun itself (A4). Same-user storage cannot protect itself from same-user code. The principled remedy is to move approvals and state behind a broker running as a different user, with the agent host's own file and command restrictions guarding the checkout in the meantime.

Credential-store reachability. Keychain items created through `/usr/bin/security` are readable by any same-user process that invokes it, and the legacy file store keeps its key beside its ciphertext. Both are consistent with the A3 exclusion but matter for A2 with an unrestricted shell.

Audit truncation and vault rollback. The hash chain and the vault manifest MAC detect modification but not replacement by an older valid state. Anchoring the latest chain head and vault revision in the OS credential manager would give monotonicity against everything short of A3.

Partial and structured secrets. A short secret (<24 B) printed partially—say its first seven characters—is not fragment-indexed, and a structured secret's boilerplate windows cause over-redaction (Table 4). Both trace to treating every 12-byte window alike; weighting windows by estimated entropy, or decomposing URLs into their userinfo component, are natural refinements. Separately, when two distinct registered values overlap in the output, the leftmost match wins and up to eleven bytes of the second can survive; we have only reproduced this with hand-built values.

Platform. macOS's `sandbox-exec` is deprecated though functional; Windows is unsupported. Provider-side retention of anything that *did* reach context is outside ironrun's reach.

9 Related Work

Prompt injection and agent security. Indirect prompt injection [33, 60] and its formalization and benchmarks [21, 48, 78] establish that an agent's behavior can be steered by data it reads, and adaptive attacks defeat most model-level defenses [55]. Willison's "lethal trifecta" [72] and Meta's "rule of two" [51] frame exfiltration as the conjunction of private data, untrusted input, and an outbound channel. ironrun removes the first from the agent's view and, with network isolation, the third from the child.

System-level defenses. CaMeL [22], design patterns for injection-resistant agents [11], information-flow control [18, 73], privilege policies [67], and execution isolation [74], building on the dual-LLM pattern [71], constrain what a compromised planner can do. ironrun is narrower and deterministic: one data class (credentials), one sink (tool output), no change to the model or planner. Its redactor is a fixed declassification rule at a single boundary, and its strict mode corresponds to the action-selector pattern at the shell level. A recent systematization of execution security for coding agents finds deny-lists failing at high rates [64], consistent with our allowlist-first design.

MCP security. Measurement and attack studies of the MCP ecosystem [15, 34–36, 79, 80] document tool poisoning [42, 70], credential theft through MCP-connected tools [63], bidirectional data-flow risks [37], and context leaking into tool arguments of coding agents [76]. ironrun sits on the return path of a local stdio server and assumes a possibly poisoned agent.

Secret leakage and detection. Secrets leak widely in public code [50], code models can memorize them [38], and vendor data suggest AI-assisted commits leak more often than average [28, 30]. Pattern-based detectors trade precision for recall [9, 10]; ironrun sidesteps detection by matching only values it injected. Closest in the literature, Kenney et al. evaluate a vault-mediated boundary for API connectors and conclude that output redaction is a separate, necessary layer [43], and SlotGuard substitutes *detected* secrets at the transcript boundary [75]. Chen et al. identify stdout-to-context as the dominant leak path in agent skills without proposing a runtime fix [16].

CI log masking. The GitHub Actions runner masks registered secrets and automatically derived forms—base64 at all three alignments, URI, JSON, XML, and shell escapes—in job logs [31], while noting that masking relies on exact matches and is not guaranteed [32]. ironrun adopts this idea for an interactive, streaming setting and adds whitespace-tolerant and fragment matching; we also found that its own integration with GitHub's line-oriented mask command could leak multi-line values (R1).

Industry tools. Run wrappers inject secrets [2, 24, 41], and `op run` also conceals them in output. Agent access products broker credentials [1, 12]; Doppler's MCP server deliberately exposes values to agents [23] and Infisical's can mask values in its own responses [39]. Proxy substitution keeps values out of processes for HTTP(S) [7, 20, 26, 40, 58]. The nearest tools to ironrun are fnox's MCP `exec`, which injects secrets and redacts their literal values from output [27], and Prismor's Cloak, which substitutes placeholders through agent-host hooks and scrubs literal values [61]. Pattern-based scanners and gateways inspect tool traffic [29, 62, 65, 69], and other tools scrub traces or histories after the fact [45, 54]. Agent hosts now expose hooks that can rewrite tool output [5] and ship their own sandboxes [4, 7]. ironrun differs in combining encoding-, alignment-, whitespace-, and fragment-aware known-value redaction with session-bound authorization and OS containment for arbitrary programs, and in evaluating that redaction adversarially.

Isolation and integrity primitives. Heavier isolation via user-space kernels or microVMs [3, 77] and unprivileged sandboxes [17] would strengthen containment at a cost; ironrun uses namespaces, seccomp, and Seatbelt to stay a single dependency-free binary. Release signing follows Sigstore [57].

10 Conclusion

Coding agents need to use credentials far more often than they need to know them. ironrun filters known secret values and supported encodings from intercepted child-process output. Its protection is limited by matching thresholds, transformations, and the host-enforced execution boundary. A streaming known-value redactor with a provable chunking property, session-bound human authorization with in-place continuation, and fail-closed containment together reduce common accidental-disclosure paths at a cost of well under a millisecond of redaction per command. Our self-audit shows where such systems actually break—in the rendering of what humans approve, in surfaces built for humans that agents can also reach, and in the structure of encodings—and our

measurements make the boundary of known-value redaction explicit. ironrun is available under the MIT license at https://github.com/Generalized-Labs/ironrun.

Availability. The evaluation harness (`go run ./paper/eval`) runs current-implementation leakage, false-positive, and performance experiments; `make -C paper` builds this document. The procedure in `paper/REPRODUCING.md` copies the same checked harness into the pinned v0.5.0 baseline. A Linux/Go 1.26.9 rerun reproduced 52/126 baseline and 28/126 current threshold cases, including 21/72 and 3/72 accidental cases. Raw outputs, source hashes, and host metadata are provided under `docs/validation/2026-10-20/`. Historical timing, false-positive, self-audit, and RQ4 results still require their separate supporting records; the leakage rerun does not validate those claims. The current source must be pinned to a final commit before publication.

Disclosure. Portions of the audit and of this manuscript's preparation were assisted by LLM-based tools under the authors' direction; all findings were confirmed by executed probes or manual review, and all claims were checked against the code.

References

[1] 1Password. 1password launches unified access platform to help companies securely deploy AI agents. Press release, March 2026. URL https://1password.com/press/2026/mar/1password-unified-access. Published 2026-03-17. Accessed 2026-10-09.

[2] 1Password. `op run` — 1password CLI reference, 2026. URL https://www.1password.dev/cli/reference/commands/run/. Accessed 2026-10-09.

[3] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020. URL https://www.usenix.org/conference/nsdi20/presentation/agache.

[4] Anthropic. sandbox-runtime: lightweight os-level filesystem and network sandboxing for arbitrary processes. GitHub repository anthropics/sandbox-runtime, 2025. URL https://github.com/anthropics/sandbox-runtime. Accessed 2026-10-09.

[5] Anthropic. Hooks reference (posttooluse `updatedToolOutput`). Claude Code documentation, 2026. URL https://code.claude.com/docs/en/hooks. Accessed 2026-10-09.

[6] Anthropic. Configure permissions. Claude Code documentation, 2026. URL https://code.claude.com/docs/en/permissions. Accessed 2026-10-09.

[7] Anthropic. Configure the sandboxed bash tool. Claude Code documentation, 2026. URL https://code.claude.com/docs/en/sandboxing. Sections "Protect credentials" and "Mask credentials". Accessed 2026-10-09.

[8] Apple Inc. sandbox-exec(1) — execute within a sandbox (DEPRECATED). macOS General Commands Manual, 2026. Checked on macOS 26.4; accessed 2026-10-09.

[9] Setu Kumar Basak, Jamison Cox, Bradley Reaves, and Laurie Williams. A comparative study of software secrets reporting by secret detection tools. In *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12, 2023. doi: 10.1109/ESEM56168.2023.10304853.

[10] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A dataset of software secrets. In *IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 347–351, 2023. doi: 10.1109/MSR59073.2023.00053.

[11] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. URL https://arxiv.org/abs/2506.08837.

[12] Bitwarden. Shadow AI agents are already in your organization. here's how to secure their credential access with agent access SDK. Bitwarden blog, June 2026. URL https://bitwarden.com/blog/shadow-ai-agents-how-to-secure-credential-access-with-agent-access-sdk/. Code: https://github.com/bitwarden/agent-access. Accessed 2026-10-09.

[13] Dionysus Blazakis. The apple sandbox. Black Hat DC 2011 (whitepaper), 2011. URL https://media.blackhat.com/bh-dc-11/Blazakis/BlackHat_DC_2011_Blazakis_Apple_Sandbox-wp.pdf. Accessed 2026-10-09.

[14] castours-dev. SECURITY: Claude code reveals secret keys and credentials in terminal output despite explicit prohibitions. GitHub issue anthropics/claude-code#32523, 2026. URL https://github.com/anthropics/claude-code/issues/32523. Opened 2026-03-09; closed. Accessed 2026-10-09.

[15] Pei Chen, Baichao An, Mengying Wu, Binwang Wan, Geng Hong, Jinsong Chen, Xudong Pan, Jiarun Dai, and Min Yang. Rethinking MCP security: A large-scale study of runtime MCP servers and security scanner reliability, 2026. URL https://arxiv.org/abs/2607.11086.

[16] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. How your credentials are leaked by LLM agent skills: An empirical study, 2026. URL

https://arxiv.org/html/2604.03070v2. Version 2, 19 June 2026. Accessed 2026-10-20.

[17] containers/bubblewrap contributors. bubblewrap: low-level unprivileged sandboxing tool. GitHub repository, 2016. URL https://github.com/containers/bubblewrap. Accessed 2026-10-09.

[18] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025. URL https://arxiv.org/abs/2505.23643.

[19] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*, 2009. URL https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident.

[20] CyberArk. Secretless broker. GitHub repository cyberark/secretless-broker, 2017. URL https://github.com/cyberark/secretless-broker. Accessed 2026-10-09.

[21] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[22] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL https://arxiv.org/abs/2503.18813.

[23] Doppler. Doppler MCP server. GitHub repository DopplerHQ/mcp-server, 2025. URL https://github.com/DopplerHQ/mcp-server. Experimental. Accessed 2026-10-09.

[24] Doppler. CLI guide (`doppler run`), 2026. URL https://docs.doppler.com/docs/cli. Accessed 2026-10-09.

[25] Martin Paul Eve. Claude code can consume, transmit, and compromise your .env files even if you tell it not to. Blog post, April 2026. URL https://eve.gd/2026/04/19/claude-code-can-consume-transmit-and-compromise-your-env-files-even-if-you-tell-it-not-to/. Accessed 2026-10-09.

[26] Fly.io. tokenizer: HTTP proxy that injects 3rd party credentials into requests. GitHub repository superfly/tokenizer, 2023. URL https://github.com/superfly/tokenizer. Accessed 2026-10-09.

[27] fnox project. fnox: MCP server guide. Documentation for jdx/fnox, 2026. URL https://fnox.jdx.dev/guide/mcp.html. Repository: https://github.com/jdx/fnox. Accessed 2026-10-09.

[28] GitGuardian. The state of secrets sprawl 2025, 2025. URL https://www.gitguardian.com/state-of-secrets-sprawl-report-2025. Accessed 2026-10-09.

[29] GitGuardian. ggshield secret scan ai-hook. ggshield documentation, 2026. URL https://docs.gitguardian.com/ggshield-docs/reference/secret/scan/ai-hook. Accessed 2026-10-09.

[30] GitGuardian. The state of secrets sprawl 2026, 2026. URL https://www.gitguardian.com/state-of-secrets-sprawl-report-2026. 5th edition (covers 2025 data). Accessed 2026-10-09.

[31] GitHub. actions/runner: `ValueEncoders.cs` (secret-masker value encoders). Source code, 2019. URL https://github.com/actions/runner/blob/main/src/Sdk/DTLogging/Logging/ValueEncoders.cs. Registered in `src/Runner.Common/HostContext.cs`; present since the initial commit, 2019-10-10. Accessed 2026-10-09.

[32] GitHub. Secure use reference (GitHub actions). GitHub Docs, 2026. URL https://docs.github.com/en/actions/reference/security/secure-use. Accessed 2026-10-09.

[33] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985.

[34] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. A measurement study of model context protocol ecosystem, 2025. URL https://arxiv.org/abs/2509.25292.

[35] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers, 2025. URL https://arxiv.org/abs/2506.13538.

[36] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions, 2025. URL https://arxiv.org/abs/2503.23278.

[37] Xinyi Hou, Yanjie Zhao, and Haoyu Wang. Unsafe by flow: Uncovering bidirectional data-flow risks in MCP ecosystem, 2026. URL https://arxiv.org/abs/2605.07836.

[38] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials. *Proceedings of the ACM on Software Engineering*, 1(FSE):2515–2537, 2024. doi: 10.1145/3660818.

[39] Infisical. Infisical MCP server. GitHub repository Infisical/infisical-mcp-server, 2025. URL https://github.com/Infisical/infisical-mcp-server. See INFISICAL_MASK_SECRET_VALUES. Accessed 2026-10-09.

[40] Infisical. Agent proxy. Infisical documentation, 2026. URL https://infisical.com/docs/documentation/platform/agent-proxy/overview. Accessed 2026-10-09.

[41] Infisical. CLI reference: infisical run, 2026. URL https://infisical.com/docs/cli/commands/run. Accessed 2026-10-09.

[42] Invariant Labs. MCP security notification: Tool poisoning attacks, April 2025. URL https://invariantlabs.ai/blog/mcp-security-notification. Published 2025-04-01. Accessed 2026-10-09.

[43] Patrick Kenney, Hadi Ahmadi, Denis Lusson, Donald Nguyen, and Gurbinder Gill. API secrets should never become tokens in the LLM's vocabulary: A threat analysis of API credential handling in LLM agent systems and an empirical evaluation of a vault-mediated execution boundary, 2026. URL https://arxiv.org/abs/2609.33371.

[44] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. Characterizing the security of GitHub CI workflows. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022. URL https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev.

[45] LangWatch. Secrets redaction in coding agent traces, 2026. URL https://langwatch.ai/docs/coding-agents/secrets-redaction. Accessed 2026-10-09.

[46] Linux man-pages project. network_namespaces(7) — overview of linux network namespaces, 2026. URL https://man7.org/linux/man-pages/man7/network_namespaces.7.html. Accessed 2026-10-09.

[47] Linux man-pages project. user_namespaces(7) — overview of linux user namespaces, 2026. URL https://man7.org/linux/man-pages/man7/user_namespaces.7.html. Accessed 2026-10-09.

[48] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024. URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei.

[49] Jaroslav Lobačevski. Keeping your GitHub actions and workflows secure part 1: Preventing pwn requests. GitHub Security Lab, August 2021. URL https://securitylab.github.com/resources/github-actions-preventing-pwn-requests/. Accessed 2026-10-09.

[50] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public GitHub repositories. In *Network and Distributed System Security Symposium (NDSS)*, 2019. doi: 10.14722/ndss.2019.23418.

[51] Meta AI. Agents rule of two: A practical approach to AI agent security. Meta AI blog, October 2025. URL https://ai.meta.com/blog/practical-ai-agent-security/. Published 2025-10-31. Accessed 2026-10-09.

[52] Model Context Protocol. Model context protocol specification (version 2026-07-28), 2026. URL https://modelcontextprotocol.io/specification/2026-07-28. Accessed 2026-10-09.

[53] Dor Munis. Claude code automatically loads .env secrets, without telling you. Knostic blog, December 2025. URL https://www.knostic.ai/blog/claude-loads-secrets-without-permission. Published 2025-12-08. Accessed 2026-10-09.

[54] Ishan Naik. agent-sweep: Find and redact secrets in AI coding agent histories. GitHub repository Ishannaik/agent-sweep, 2026. URL https://github.com/Ishannaik/agent-sweep. Accessed 2026-10-09.

[55] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections, 2025. URL https://arxiv.org/abs/2510.09023.

[56] nbyk115. [feature request] allow a posttooluse hook to modify (redact) the output string. GitHub issue anthropics/claude-code#64326, 2026. URL https://github.com/anthropics/claude-code/issues/64326. Title translated from Japanese. Opened 2026-05-31; closed. Accessed 2026-10-09.

[57] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, pages 2353–2367, 2022. doi: 10.1145/3548606.3560596.

[58] OneCLI. Onecli: open-source sandboxed agent harness with credential-injecting gateway. GitHub repository onecli/onecli, 2026. URL https://github.com/onecli/onecli. Accessed 2026-10-09.

[59] Alexander Panfilov, David Schmotz, Ilia Shumailov, Luca Beurer-Kellner, Joachim Schaeffer, Ameya Prabhu, Jonas Geiping, and Maksym Andriushchenko. Stealing reasoning traces from proprietary LLM APIs, 2026. URL https://arxiv.org/abs/2608.09867.

[60] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. URL https://arxiv.org/abs/2211.09527.

[61] PrismorSec. Prismor: self-hosted runtime control plane for AI agents (sweep and cloak). GitHub repository (formerly immunity-agent), 2026. URL https://github.com/PrismorSec/prismor. Accessed 2026-10-09.

[62] r3352. redact-mcp: MCP server that auto-obfuscates sensitive data. GitHub repository, 2026. URL https://github.com/r3352/redact-mcp. Accessed 2026-10-09.

[63] Brandon Radosevich and John Halloran. MCP safety audit: LLMs with the model context protocol allow major security exploits, 2025. URL https://arxiv.org/abs/2504.03767.

[64] Mohammadreza Rashidi. The balkanization of execution-security research for AI coding agents: Isolation, access control, and time-of-check-to-time-of-use vulnerabilities, 2026. URL https://arxiv.org/abs/2607.05743.

[65] sainitish1609. mcp-guard: local security firewall and secret sanitizer for MCP. GitHub repository, 2026. URL https://github.com/sainitish1609/mcp-guard. Accessed 2026-10-09.

[66] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. doi: 10.1145/317087.317089.

[67] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. URL https://arxiv.org/abs/2504.11703. v1/v2 titled "Progent: Programmable Privilege Control for LLM Agents".

[68] The Linux Kernel documentation. Seccomp BPF (SECure COMPuting with filters), 2026. URL https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html. Accessed 2026-10-09.

[69] TrueFoundry. AI gateway: Secrets detection guardrail, 2026. URL https://www.truefoundry.com/docs/ai-gateway/secrets-detection. Accessed 2026-10-09.

[70] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A benchmark for tool poisoning on real-world MCP servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 35811–35819, 2026. doi: 10.1609/aaai.v40i42.40895.

[71] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. Blog post, April 2023. URL https://simonwillison.net/2023/Apr/25/dual-llm-pattern/. Accessed 2026-10-09.

[72] Simon Willison. The lethal trifecta for AI agents: private data, untrusted content, and external communication. Blog post, June 2025. URL https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/. Accessed 2026-10-09.

[73] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024. URL https://arxiv.org/abs/2409.19091.

[74] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.241131.

[75] Haocheng Xia and Yongjoo Park. SlotGuard: Stop oversharing private local context in LLM agent transcripts. In *ICML 2026 Workshop (AIWILD)*, 2026. URL https://arxiv.org/abs/2607.17147.

[76] Yuchong Xie, Mingyu Luo, Zesen Liu, Zhixiang Zhang, Kaikai Zhang, Yu Liu, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. Red-teaming coding agents from a tool-invocation perspective: An empirical security assessment, 2025. URL https://arxiv.org/abs/2509.05755.

[77] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, 2019. URL https://www.usenix.org/conference/hotcloud19/presentation/young.

[78] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL https://aclanthology.org/2024.findings-acl.624/.

[79] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *IEEE Symposium on Security and Privacy (S&P)*, 2026. URL https://arxiv.org/abs/2509.06572.

[80] Huijun Zhou, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang. A first measurement study on authentication security in real-world remote MCP servers, 2026. URL https://arxiv.org/abs/2605.22333.